SE 4111

Lab 7: GitHub Actions CI

Dr. Mawal A. Mohammed

© Mawal A. Mohammed

Lab 7

© Mawal A. Mohammed

Lab 7 - Learning Objectives

- Students will be able to:
  - Understand Continuous Integration/Continuous Deployment
  - Create GitHub Actions workflows
  - Automate build and test on every push
  - Configure workflow triggers and jobs
  - View and debug workflow results

© Mawal A. Mohammed

Lab 7 - Introduction

- What is CI/CD?
  - **CI**: Continuous Integration - automatically build and test code (this lab)
  - **CD**: Continuous Deployment - automatically deploy to production



- Benefits:
  - Catch bugs early (automated testing)
  - Consistent builds across team
  - Faster feedback on code changes
  - Automated quality gates
- GitHub Actions:
  - Built-in CI/CD for GitHub repositories
  - Free for public repositories

© Mawal A. Mohammed

Lab 7 - How GitHub Actions Works

- **Workflow**: Automated process defined in YAML file
  - Located in .github/workflows/ directory
- **Trigger**: Event that starts the workflow
  - Push, pull request, schedule, manual
- **Job**: Set of steps running on same runner
  - Jobs can run in parallel or sequence
- **Step**: Individual task in a job
  - Run commands or use actions
- **Runner**: Virtual machine that executes jobs
  - Ubuntu, Windows, macOS available

© Mawal A. Mohammed

Lab 7 - The Project: a Calculator class

The full project code and files are associated with the lab.

- A tiny Java class with a single add(a, b) method – this is what CI will build and test

```
// src/main/java/com/example/Calculator.java
package com.example;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }
}
```

© Mawal A. Mohammed

Lab 7 - The Test: CalculatorTest

- These three JUnit 5 tests will be executed by GitHub Actions on every push

```
// src/test/java/com/example/CalculatorTest.java
package com.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculatorTest {

    @Test
    void testAddPositiveNumbers() {
        Calculator calc = new Calculator();
        assertEquals(5, calc.add(2, 3));
    }

    @Test
    void testAddWithZero() {
        Calculator calc = new Calculator();
        assertEquals(7, calc.add(7, 0));
    }

    @Test
    void testAddNegativeNumbers() {
        Calculator calc = new Calculator();
        assertEquals(-8, calc.add(-3, -5));
    }
}
```

© Mawal A. Mohammed

Lab 7 - Minimal pom.xml

- Just enough Maven to compile, test, and package the Calculator

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>calculator-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

© Mawal A. Mohammed

Lab 7 - The Workflow File

- This workflow runs on every push and pull request to main

```
# Create this file in your repo:
# .github/workflows/ci.yml

name: Java CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      - name: Compile the project
        run: mvn -B clean compile

      - name: Run unit tests
        run: mvn -B test
```

© Mawal A. Mohammed

Lab 7 - The Workflow File (Cont.)

- After the run finishes, download the JAR and reports from the Actions page

```
# Add these steps after 'Run unit tests' in ci.yml:

      - name: Package the application
        run: mvn -B package -DskipTests

      - name: Upload test reports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-reports
          path: target/surefire-reports/

      - name: Upload built JAR
        uses: actions/upload-artifact@v4
        with:
          name: calculator-jar
          path: target/*.jar
```

© Mawal A. Mohammed

Lab 7 - Reading the Workflow Line by Line

- name: 'Java CI' - shown in the Actions tab
- on: push / pull_request - triggers the workflow
- jobs.build-and-test - one job, named 'build-and-test'
- runs-on: ubuntu-latest - the runner OS
- steps: an ordered list of actions
    - actions/checkout@v4 - clones your repo into the runner
    - actions/setup-java@v4 - installs Temurin JDK 17
    - cache: maven - caches This tells GitHub to remember what it downloaded last time so future runs are much faster
    - mvn -B compile / mvn -B test - the actual build commands
- -B = batch mode (no interactive prompts in the log)

© Mawal A. Mohammed

Lab 7 - Activity 1: Set Up the Repository

- Create the project locally (download the project and extract it):
- Verify it builds on your machine FIRST:
  - mvn clean test
  - All 3 tests should pass before automating anything
- Create a new GitHub repository named 'calculator-app'
- Push the code:
  - git init && git add . && git commit -m "initial commit"
  - git branch -M main
  - git remote add origin https://github.com/<your_username>/calculator-app.git
  - git push -u origin main

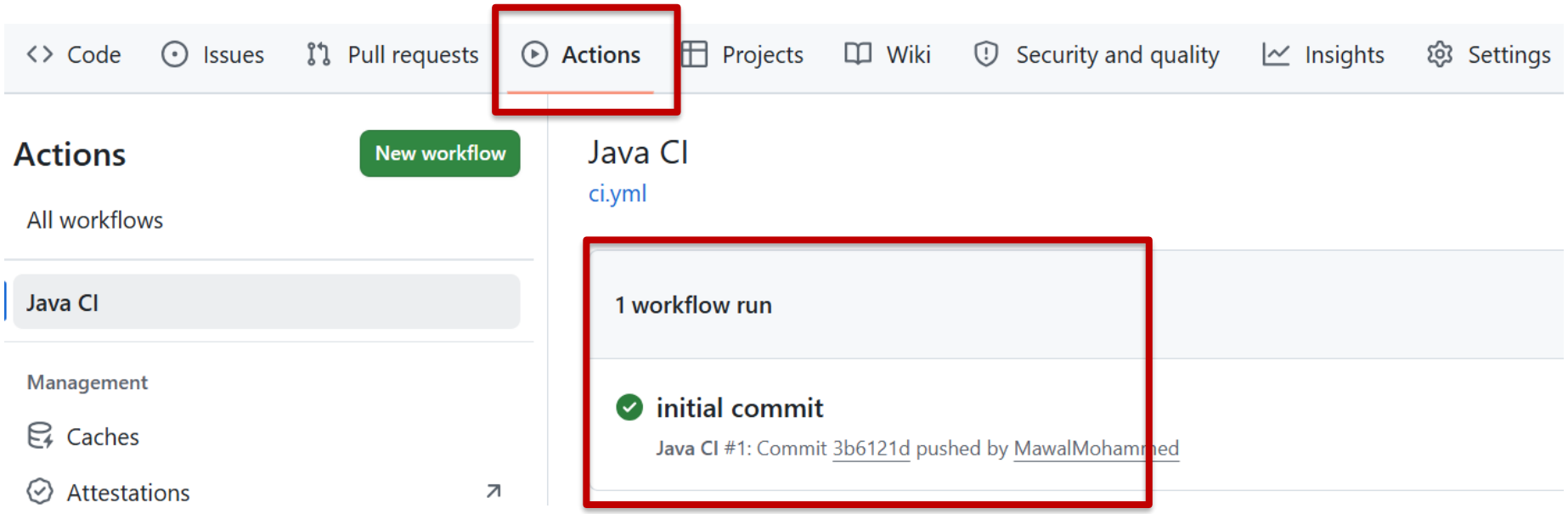
© Mawal A. Mohammed

Lab 7 – Reading the results

- Open the repository on github.com and click the Actions tab (update if your already in the page)
- You should see a run named after your commit message
  - Yellow dot = in progress
  - Green check = success - all tests passed
  - Red X = failure - open the run to see which step failed
- Click the job 'build-and-test' to expand each step's logs

© Mawal A. Mohammed

Lab 7 – Github actions Page



© Mawal A. Mohammed

Lab 7 - Activity 2: Make the Pipeline Fail (on purpose)

- CI's real value: a failing test on main is impossible to ignore

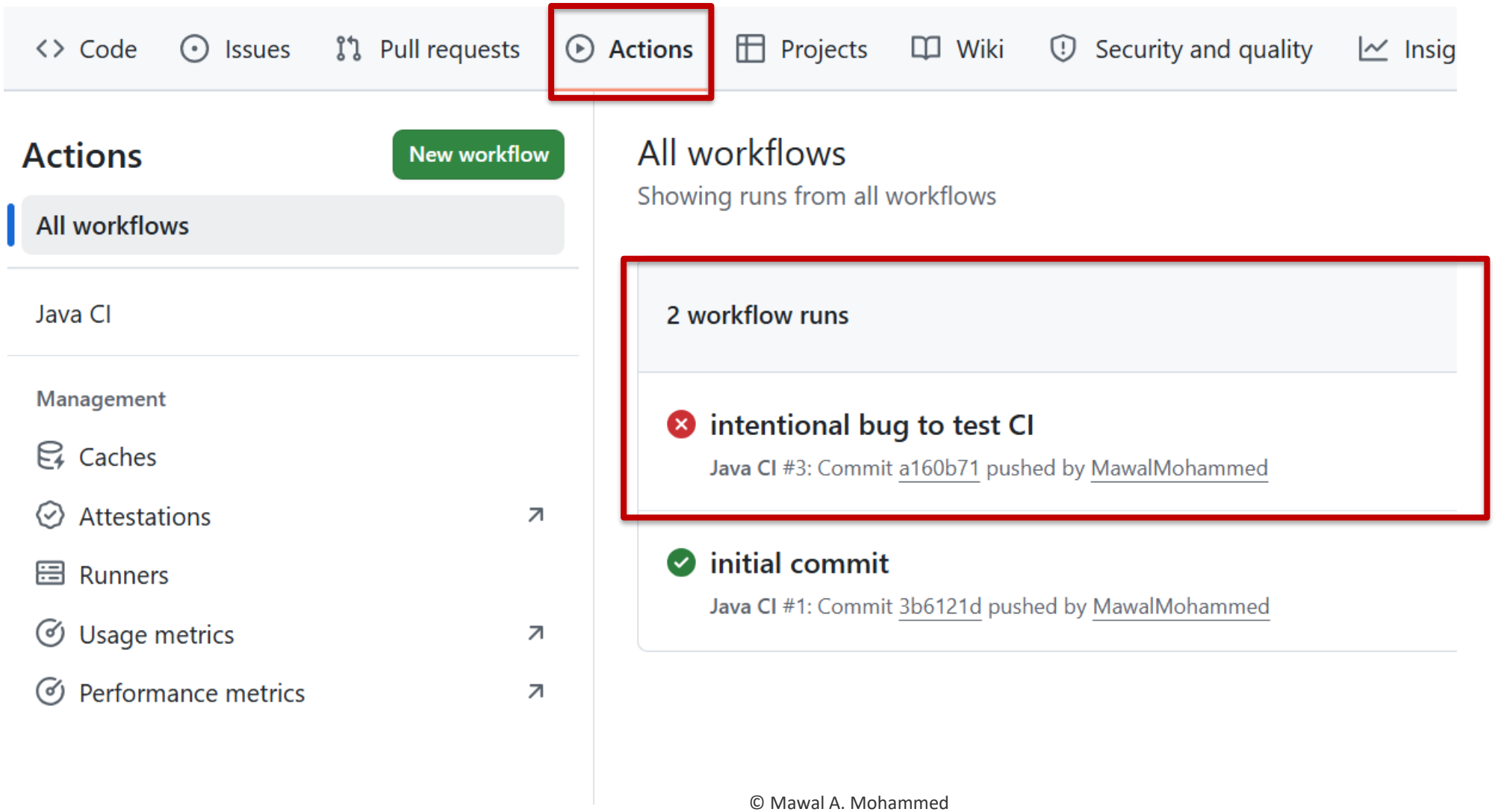
```
// Edit Calculator.java and break add() intentionally:
public int add(int a, int b) {
    return a - b;   // BUG: was a + b
}

// Commit and push the change:
git add src/main/java/com/example/Calculator.java
git commit -m "intentional bug to test CI"
git push

// In the Actions tab:
//   - The run goes RED
//   - Open the failing 'Run unit tests' step
//   - You will see exactly which assertEquals failed
```

© Mawal A. Mohammed

Lab 7 – Github actions Page



© Mawal A. Mohammed

Lab 7 - Summary

- GitHub Actions automates compile, test, and package on every push
- Workflows are YAML files in .github/workflows/
- Triggers (on:), jobs, steps, runners - the four building blocks
- actions/checkout + actions/setup-java are enough for a Java CI
- Cache Maven (cache: maven) to make repeat runs fast
- Upload artifacts to download the built JAR and test reports
- A failed test on main shows up immediately as a red X in Actions
- You now have a tiny but real CI pipeline for the Calculator project

© Mawal A. Mohammed